



INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO



INGENIERÍA EN SISTEMAS COMPUTACIONALES

MATERIA:

Aplicaciones para Comunicaciones en Red

TEMA:

**Práctica: Transferencia de Archivos con Go-Back-N
sobre UDP**

ALUMNO:

**Erreguin Franco Yair Alejandro
Boleta: 2022630121**

PROFESOR:

Moreno Cervantes Axel Ernesto

26 de junio de 2026

Índice

1. Introducción	2
1.1. UDP: potencia sin garantías	2
1.2. Ventana Deslizante	2
1.3. Go-Back-N (GBN)	3
1.4. Fragmentación de Archivos	3
2. Desarrollo Técnico	4
2.1. La Clase Paquete: el sobre de cada envío	4
2.2. Servidor: fragmentación y ventana deslizante GBN	5
2.2.1. Fragmentación	5
2.2.2. Ventana deslizante y reenvío GBN	5
2.3. Cliente: recepción en orden y ensamblado	6
2.3.1. Recepción con control de orden	6
2.3.2. Ensamblado del archivo	7
3. Flujo Completo de la Transferencia	8
4. Ejecución y Resultados	8
4.1. Ejecución	8
4.2. Resultados	9
5. Conclusiones	10
6. Referencias	11

1. Introducción

Transferir un archivo de un punto a otro a través de la red parece algo sencillo, pero en cuanto se elige un protocolo de transporte sin garantías, como UDP, el problema se vuelve interesante: ¿cómo asegurarse de que todos los datos llegan, en orden, sin huecos?

El objetivo de esta práctica es implementar esa garantía de forma manual, usando el protocolo **Go-Back-N (GBN)**, que es uno de los esquemas clásicos de control de flujo basados en ventana deslizante. La aplicación transfiere canciones en formato MP3 de un servidor a un cliente, fragmentando el archivo en pedazos pequeños, numerándolos, y usando ACKs y timeouts para asegurar que todo llegue completo y en el orden correcto.

1.1. UDP: potencia sin garantías

UDP (*User Datagram Protocol*) es un protocolo de transporte minimalista. Cuando el servidor envía un datagrama, lo lanza a la red y no sabe si llegó, si llegó en orden, ni si llegó íntegro. No hay conexión, no hay reenvío automático, no hay nada.

Esto lo hace muy rápido y eficiente, pero delega toda la responsabilidad de confiabilidad al programador. Por eso es el protocolo de elección cuando se quiere implementar un mecanismo de control propio, como es el caso de esta práctica con GBN.

En Java, un socket UDP se crea y se usa así:

```
1 // Servidor: escucha en un puerto fijo
2 DatagramSocket socket = new DatagramSocket(9876);
3
4 // Enviar un datagrama
5 byte[] raw = paquete.aBytes();
6 socket.send(new DatagramPacket(raw, raw.length, addr, puerto));
7
8 // Recibir un datagrama
9 byte[] buf = new byte[65535];
10 DatagramPacket dp = new DatagramPacket(buf, buf.length);
11 socket.receive(dp); // bloqueante hasta que llegue algo
```

Listing 1: Creación y uso básico de un socket UDP

Cada llamada a `send()` envía un datagrama independiente. Cada llamada a `receive()` recibe uno. No hay orden garantizado entre ellos.

1.2. Ventana Deslizante

La ventana deslizante es la técnica que permite enviar varios paquetes sin esperar un ACK por cada uno. En lugar de hacer el ciclo: *envío 1* $\rightarrow$ *espero ACK* $\rightarrow$ *envío 2* $\rightarrow$ *espero ACK*..., el emisor mantiene un rango de paquetes que puede enviar sin confirmación. Ese rango es la **ventana**.

La ventana tiene dos bordes:

- **base**: el paquete más antiguo que aún no fue confirmado.
- **base + N**: el límite superior de la ventana, donde N es el tamaño máximo.

Cuando llega un ACK, la ventana *desliza* hacia adelante, liberando espacio para enviar nuevos paquetes. Esto mantiene el canal de red ocupado la mayor parte del tiempo, en lugar de estar inactivo esperando confirmaciones una por una.

```

Paquetes:  0   1   2   3   4   5   6   7   ...
            [       ventana (N=4)       ]
            ^base                        ^base+N
                                   ^sig (próximo a enviar)

```

1.3. Go-Back-N (GBN)

Go-Back-N es una política específica de ventana deslizante que define qué hacer cuando algo sale mal. Sus dos reglas fundamentales son:

1. **El receptor acepta solo paquetes en orden estricto.** Si llega el paquete 3 pero el receptor esperaba el 2, lo descarta silenciosamente. No envía ACK ni NAK, simplemente lo ignora.
2. **Si el emisor no recibe ACK antes de que expire su timer, reenvía *todos* los paquetes desde el inicio de la ventana.** De ahí el nombre: retrocede y vuelve a enviar desde **base**.

Esta simplicidad en el receptor (no necesita guardar paquetes fuera de orden) tiene un costo: ante cualquier pérdida, se reenvían múltiples paquetes aunque algunos ya hubieran llegado bien. Es el trade-off del protocolo.

1.4. Fragmentación de Archivos

Un archivo MP3 puede pesar varios megabytes, pero un datagrama UDP tiene un tamaño máximo práctico de alrededor de 64 KB (y por debajo de la MTU de Ethernet, idealmente menos de 1500 bytes). Por eso el archivo debe **fragmentarse**: dividirse en trozos pequeños, numerarlos secuencialmente, enviarlos de forma independiente, y al final **ensamblarse** en el receptor pegándolos en el orden correcto.

En esta práctica cada fragmento mide 1024 bytes (1 KB), salvo el último que puede ser menor.

2. Desarrollo Técnico

2.1. La Clase Paquete: el sobre de cada envío

Antes de explicar el flujo del servidor y el cliente, es importante entender `Paquete.java`. Esta clase resuelve un problema concreto: UDP solo sabe transportar `byte[]`, pero el programa necesita enviar información estructurada (tipo de mensaje, número de secuencia, datos, etc.). `Paquete` es el puente entre esas dos realidades.

Cada paquete lleva cuatro campos en su cabecera:

- **tipo** (1 byte): indica qué clase de paquete es.
- **numSec** (4 bytes): número de secuencia del fragmento.
- **total** (4 bytes): cantidad total de fragmentos del archivo.
- **datos.length** (4 bytes): tamaño del payload que sigue.

Eso suma 13 bytes fijos de cabecera, más el payload variable.

```
1 public static final byte CATALOGO = 0;
2 public static final byte PEDIR    = 1;
3 public static final byte DATOS    = 2;
4 public static final byte ACK      = 3;
5 public static final byte FIN      = 4;
6
7 // Serializar: construir el byte[] que viajar por la red
8 public byte[] aBytes() {
9     // cabecera: tipo(1) + numSec(4) + total(4) + len(4) = 13 bytes
10    ByteBuffer buf = ByteBuffer.allocate(13 + datos.length);
11    buf.put(tipo);
12    buf.putInt(numSec);
13    buf.putInt(total);
14    buf.putInt(datos.length);
15    buf.put(datos);
16    return buf.array();
17 }
18
19 // Deserializar: reconstruir el objeto desde los bytes recibidos
20 public static Paquete desdBytes(byte[] raw, int len) {
21    ByteBuffer buf = ByteBuffer.wrap(raw, 0, len);
22    byte tipo = buf.get();
23    int numSec = buf.getInt();
24    int total = buf.getInt();
25    int dLen = buf.getInt();
26    byte[] datos = new byte[dLen];
27    buf.get(datos);
28    return new Paquete(tipo, numSec, total, datos);
29 }
```

Listing 2: `Paquete.java` — Tipos y serialización

Vale la pena notar que `Paquete` no tiene un campo `longitud` explícito porque no es necesario: en el objeto Java, `datos.length` ya existe como propiedad nativa del arreglo. En el binario, esos 4 bytes se escriben al serializar y se leen al deserializar. Guardarlo como campo adicional sería información duplicada.

2.2. Servidor: fragmentación y ventana deslizante GBN

El servidor tiene dos responsabilidades: leer el archivo, partirlo en fragmentos, y enviarlos aplicando el control de flujo GBN.

2.2.1. Fragmentación

```
1 static final int CHUNK = 1024; // bytes por paquete
2
3 byte[] archivo = Files.readAllBytes(Path.of(ruta));
4
5 List<byte[]> fragmentos = new ArrayList<>();
6 for (int i = 0; i < archivo.length; i += CHUNK) {
7     int fin = Math.min(i + CHUNK, archivo.length);
8     fragmentos.add(Arrays.copyOfRange(archivo, i, fin));
9 }
10 int total = fragmentos.size();
```

Listing 3: Servidor.java — Lectura y fragmentación del archivo

El `Math.min` en el cálculo de `fin` es importante: garantiza que el último fragmento no desborde el arreglo aunque el archivo no sea múltiplo exacto de 1024 bytes.

2.2.2. Ventana deslizante y reenvío GBN

```
1 int base = 0; // inicio de la ventana (sin ACK a n)
2 int sig = 0; // siguiente paquete a enviar
3
4 while (base < total) {
5
6     // Llenar la ventana: enviar todo lo que cabe
7     while (sig < base + VENTANA && sig < total) {
8         Paquete p = new Paquete(Paquete.DATOS, sig, total,
9                                 fragmentos.get(sig));
10        enviar(socket, p, cliente, puerto);
11        sig++;
12    }
13
14    // Esperar un ACK
15    try {
16        socket.receive(ackDp);
17        Paquete ack = Paquete.desdBytes(ackDp.getData(), ackDp.getLength());
18
19        if (ack.tipo == Paquete.ACK) {
20            base = ack.numSec + 1; // la ventana avanza
```

```

21     }
22   } catch (SocketTimeoutException e) {
23     // Timeout: retroceder al inicio de la ventana y reenviar todo
24     sig = base;
25   }
26 }

```

Listing 4: Servidor.java — Bucle principal de Go-Back-N

La línea `sig = base` dentro del `catch` es donde ocurre el “Go-Back-N” propiamente dicho. Al resetear `sig` al valor de `base`, la siguiente iteración del `while` externo vuelve a llenar la ventana desde el paquete más antiguo sin confirmar, reenviando todos los paquetes que estaban en tránsito.

Cuando todos los fragmentos fueron confirmados (`base >= total`), el servidor envía un paquete FIN con el total de fragmentos como número de secuencia, para que el cliente sepa que la transferencia terminó.

2.3. Cliente: recepción en orden y ensamblado

El cliente mantiene una única variable, `esperado`, que indica el número de secuencia del próximo paquete que debe llegar. Solo acepta paquetes que coincidan exactamente con ese valor.

2.3.1. Recepción con control de orden

```

1  Map<Integer, byte[]> recibidos = new HashMap<>();
2  int esperado = 0;
3  int total    = -1;
4
5  while (true) {
6      try {
7          socket.receive(rdp);
8          Paquete pkt = Paquete.desdBytes(rdp.getData(), rdp.getLength());
9
10         if (pkt.tipo == Paquete.FIN) {
11             total = pkt.numSec; // el servidor indic cu ntos fragmentos
12                                 hay
13             break;
14         }
15
16         if (pkt.tipo == Paquete.DATOS) {
17             if (pkt.numSec == esperado) {
18                 // Paquete en orden: guardar y confirmar
19                 recibidos.put(pkt.numSec, pkt.datos);
20                 Paquete ack = new Paquete.ACK, esperado, 0,
21                               new byte[0]);
22                 enviar(socket, ack, addr, PUERTO);
23                 esperado++;
24             }
25             // Si numSec != esperado: fuera de orden, se descarta sin ACK
26         }
27     } catch (SocketTimeoutException e) {

```

```

27         break;
28     }
29 }

```

Listing 5: Cliente.java — Recepción Go-Back-N

El descarte silencioso (no enviar ACK cuando el paquete llega fuera de orden) es lo que dispara el mecanismo de reenvío en el servidor: sin ACK, el timer del servidor expira y retrocede al inicio de su ventana.

2.3.2. Ensamblado del archivo

Una vez terminada la recepción, el `HashMap` contiene todos los fragmentos indexados por su número de secuencia. Ensamblar el archivo es simplemente escribirlos en orden:

```

1 new File("downloads").mkdirs();
2 File archivo = new File("downloads/" + canciones[eleccion]);
3
4 try (FileOutputStream fos = new FileOutputStream(archivo)) {
5     for (int i = 0; i < total; i++) {
6         fos.write(recibidos.get(i)); // i=0, i=1, i=2... en orden
7     }
8 }

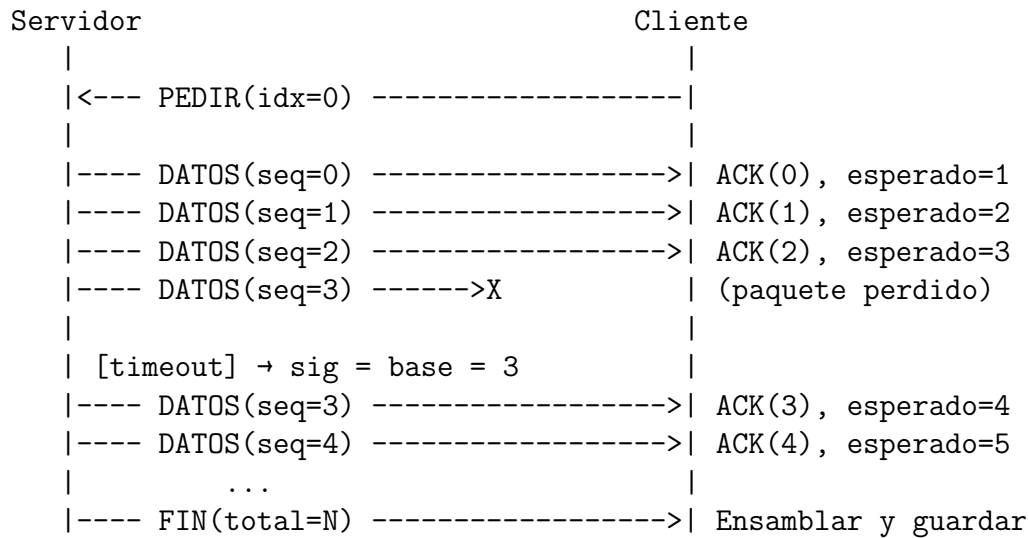
```

Listing 6: Cliente.java — Ensamblado del archivo

El resultado es el MP3 original, reconstruido byte a byte en el mismo orden en que fue fragmentado.

3. Flujo Completo de la Transferencia

El siguiente diagrama resume el intercambio de mensajes entre servidor y cliente durante una transferencia típica, incluyendo un caso de pérdida y reenvío:



La ventana de tamaño 4 permite que el servidor envíe hasta cuatro paquetes antes de necesitar un ACK. Cuando el paquete 3 se pierde, el cliente no envía ACK para ese número de secuencia ni para los siguientes fuera de orden. El servidor agota su timer, retrocede a seq=3 y reenvía desde ahí.

4. Ejecución y Resultados

4.1. Ejecución

Para llevar a cabo la ejecución se creo un shell script el cual automatiza el compilado y ejecución tanto del cliente como del servidor. A continuación se muestra el resultado de la ejecución:

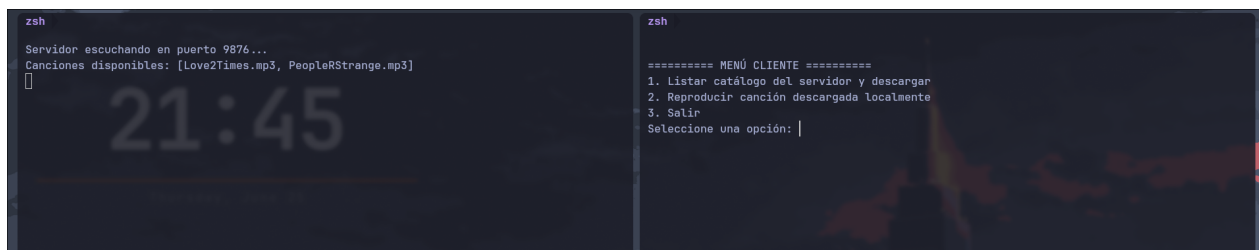
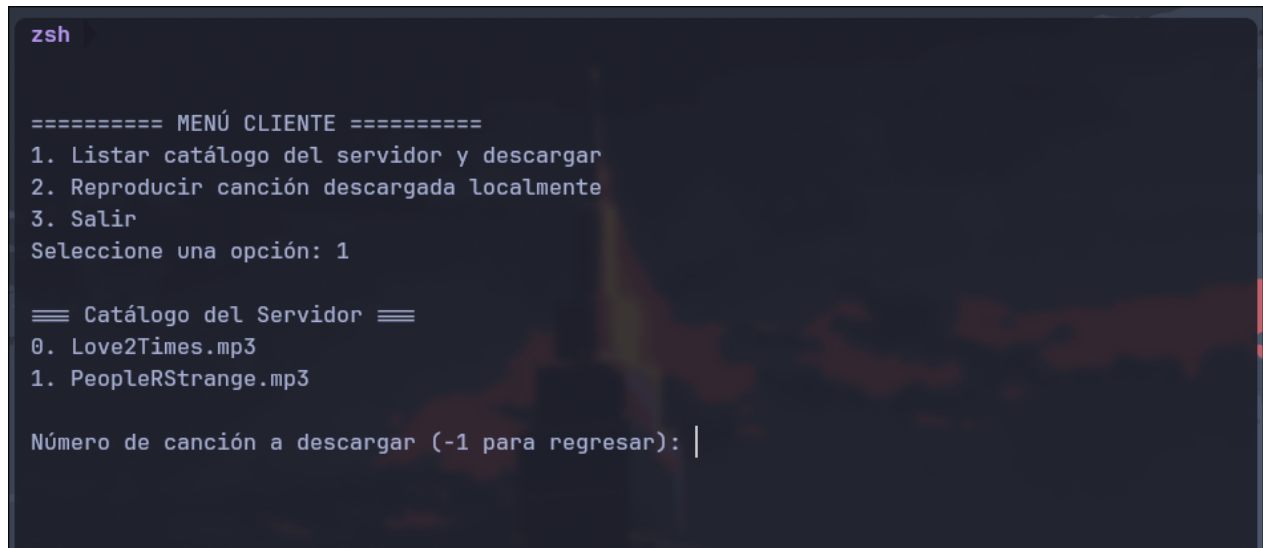


Figura 1: Log de la consola del servidor y del cliente conectado.

4.2. Resultados

El el catálogo de esta práctica se agregaron dos canciones, para descargar alguna, se hace el listado del catalogo en el cliente de modo que puede decidir que canción descargar, a continuación se muestra el listado del catalogo del servidor:

A screenshot of a terminal window with a dark background and light-colored text. The terminal shows a client menu with three options: '1. Listar catálogo del servidor y descargar', '2. Reproducir canción descargada localmente', and '3. Salir'. The user has selected option 1. Below this, a server catalog is displayed with two items: '0. Love2Times.mp3' and '1. PeopleRStrange.mp3'. At the bottom, there is a prompt 'Número de canción a descargar (-1 para regresar):' followed by a cursor.

```
zsh

===== MENÚ CLIENTE =====
1. Listar catálogo del servidor y descargar
2. Reproducir canción descargada localmente
3. Salir
Seleccione una opción: 1

=== Catálogo del Servidor ===
0. Love2Times.mp3
1. PeopleRStrange.mp3

Número de canción a descargar (-1 para regresar): |
```

Figura 2: Catalogo consultado por el cliente.

Una vez seleccionada la canción que se busca descargar entonces el server arroja un log de los paquetes enviados y la secuencia correspondiente, al finalizar el cliente arma el archivo mp3 y lo reproduce:



The image shows two terminal windows side-by-side. The left window, titled 'zsh', displays a series of network logs: 'TX seq=3103', 'ACK=3100', 'TX seq=3104', 'ACK=3101', 'TX seq=3105', 'ACK=3102', 'TX seq=3106', 'ACK=3103', 'TX seq=3107', 'ACK=3104', 'TX seq=3108', 'ACK=3105', 'TX seq=3109', 'ACK=3106', 'TX seq=3110', 'ACK=3107', 'TX seq=3111', 'ACK=3108', 'TX seq=3112', 'ACK=3109', 'TX seq=3113', 'ACK=3110', 'TX seq=3114', 'ACK=3111', 'TX seq=3115', 'ACK=3112', 'TX seq=3116', 'ACK=3113', 'TX seq=3117', 'ACK=3114', 'TX seq=3118', 'ACK=3115', 'TX seq=3119', 'ACK=3116', 'ACK=3117', 'ACK=3118', 'ACK=3119', and 'FIN enviado.' followed by a cursor. The right window, also titled 'zsh', shows a client menu: '===== MENÚ CLIENTE =====', '1. Listar catálogo del servidor y descargar', '2. Reproducir canción descargada localmente', '3. Salir', 'Seleccione una opción: 2', '== Canciones Descargadas ==', '0. Love2Times.mp3', '1. PeopleRStrange.mp3', 'Número de canción a reproducir (-1 para regresar): 1', 'Iniciando decodificación MPEG con JLayer...', 'Reproduciendo: PeopleRStrange.mp3', and a cursor.

Figura 3: Canción descargada y reproducida mediante JLayer.

5. Conclusiones

Esta práctica deja claro por qué los protocolos de confiabilidad existen y qué trabajo hacen en realidad. UDP por sí solo es un canal sin memoria y sin compromiso; todo lo que necesitamos encima de él (orden, retransmisión, detección de fin de transferencia) hay que construirlo explícitamente.

Go-Back-N es una solución elegante por su simplicidad en el receptor: no necesita buffers de reordenamiento ni lógica compleja, solo rechaza lo que llega fuera de orden y espera. El costo es que ante cualquier pérdida se reenvían múltiples paquetes, pero para redes con tasas de error bajas (como `localhost`) es más que suficiente.

La clase `Paquete` ilustra un patrón que se repite en muchos protocolos reales: definir una cabecera de tamaño fijo con campos bien posicionados, serializar a bytes para transportar, y deserializar al recibir. Es exactamente lo que hace TCP, UDP, IP o cualquier protocolo que viaja en una red.

Finalmente, el ensamblado del archivo en el cliente demuestra que un `HashMap<Integer, byte[]>` es todo lo que se necesita para reconstruir un archivo desde fragmentos desordenados, siempre que cada fragmento lleve su número de secuencia como llave.

6. Referencias

Referencias

- [1] A. S. Tanenbaum y D. Wetherall, *Computer Networks*, 5th ed. Prentice Hall, 2011.
- [2] J. F. Kurose y K. W. Ross, *Computer Networking: A Top-Down Approach*, 7th ed. Pearson, 2017.
- [3] Oracle, “DatagramSocket (Java SE docs),” 2025. [En línea]. Disponible: <https://docs.oracle.com/en/java/javase/>. [Accedido: 25-jun-2026].
- [4] J. Postel, “RFC 768 — User Datagram Protocol,” IETF, 1980. [En línea]. Disponible: <https://www.rfc-editor.org/rfc/rfc768>. [Accedido: 25-jun-2026].